Midterm 2 — Solutions

Do not distribute before the exam

DATA 412/612

2026-11-11

November 11, 2026. 75 minutes. Steps 1 to 13 are 100 points. Step 14 is 10 points of extra credit.

This file matches `midterm-2.qmd`. Accept `|>` or `%>%`. Accept a rebuilt connection when a later step needs `con`. The relative path `data/soccer.sqlite` is what the exam asks for. The path from the notes, `data/soccer/soccer.sqlite`, is also full credit.

1. Load tidyverse, DBI, RSQLite, and dbplyr. This chunk must not display messages or warnings. (5 points)

The chunk has to load tidyverse, DBI, RSQLite, and dbplyr, and it has to hide messages and warnings. A missing package is 1 point. Messages left visible are a 1-point deduction.

```
library(tidyverse)
library(DBI)
library(RSQLite)
library(dbplyr)
```

2. Download `soccer.sqlite`, the European soccer database, from David Gerard's data page, https://dcgerard.github.io/stat_412_612/data.html. Unzip it, and save `soccer.sqlite` in a folder `data` next to this file. Connect with `dbConnect()` and `SQLite()` using the relative path `data/soccer.sqlite`, and assign the connection to `con`. Print the table names, and print how many tables remain when `sqlite_sequence` is not counted. (8 points)

Seven tables remain: `Country`, `League`, `Match`, `Player`, `Player_Attributes`, `Team`, and `Team_Attributes`. Counting `sqlite_sequence` as an eighth soccer table is a 2-point deduction. An absolute path is a 1-point deduction. The file comes from https://dcgerard.github.io/stat_412_612/data.html.

```
con <- dbConnect(SQLite(), "data/soccer.sqlite")

tables <- dbListTables(con)
tables
```

```
[1] "Country"           "League"            "Match"
[4] "Player"            "Player_Attributes" "Team"
[7] "Team_Attributes"   "sqlite_sequence"
```

```
sum(tables != "sqlite_sequence")
```

```
[1] 7
```

3. Collect `Player` into a data frame. Collect `Player_Attributes` into a data frame, and keep only `player_api_id`, `potential`, and `sprint_speed`. From step 4 on, work with these data frames. (8 points)

`Player` has 11,060 rows. The attributes data frame has the same 183,978 rows and only the three named columns. Keeping every attribute column is a 1-point deduction. Leaving either table as a lazy `tbl()` is a 3-point deduction, because the later stringr steps have to run in R.

```
Player <- tbl(con, "Player") |>
  collect()

Player_Attributes <- tbl(con, "Player_Attributes") |>
  select(player_api_id, potential, sprint_speed) |>
  collect()
```

4. Prove that `player_api_id` is a primary key of `Player`, and that it is not a primary key of `Player_Attributes`. For each data frame, print the number of rows and the number of distinct `player_api_id`. In one sentence, say what those two comparisons show. (10 points)

`Player` has 11,060 rows and 11,060 distinct ids, so `player_api_id` is a primary key there. `Player_Attributes` has 183,978 rows and 11,060 distinct ids, so the id repeats and is not a primary key there. One sentence has to state both conclusions. The counts without the sentence are 8 points. Swapping the two tables is a 4-point deduction.

`player_api_id` is a primary key of `Player` because each id appears once. It is not a primary key of `Player_Attributes` because each player has many attribute rows.

```
Player |>
  summarise(n = n(), n_id = n_distinct(player_api_id))
```

```
# A tibble: 1 x 2
      n  n_id
  <int> <int>
1 11060 11060
```

```
Player_Attributes |>
  summarise(n = n(), n_id = n_distinct(player_api_id))
```

```
# A tibble: 1 x 2
       n  n_id
   <int> <int>
1 183978 11060
```

5. From `Player`, keep `player_api_id` and `player_name`. (5 points)

Two columns: `player_api_id` and `player_name`.

```
players <- Player |>
  select(player_api_id, player_name)
```

6. Keep the simple names. A simple name has exactly two uppercase letters and exactly one space. Use `str_count()` for both counts. (8 points)

9,232 names. Both counts have to use `str_count()`. One count done with `nchar()` or a base-R pattern is a 2-point deduction.

```
simple <- players |>
  filter(
    str_count(player_name, "[A-Z]") == 2,
    str_count(player_name, " ") == 1
  )

nrow(simple)
```

```
[1] 9232
```

7. Split `player_name` on the space into `first_name` and `family_name`, with `separate()`. (6 points)

`separate()` on the single space. `str_split_fixed()` or `separate_wider_delim()` is full credit. Dropping `player_api_id` is a 1-point deduction.

```
simple <- simple |>
  separate(player_name, into = c("first_name", "family_name"), sep = " ")
```

8. From the data frame in step 7, build `initials` with `str_sub()` and `str_c()`: the first letter of `first_name`, a period, the first letter of `family_name`, and a period. `Luis Martinez` becomes `L.M.`. Show the five initials that occur most often, with their counts, largest count first. Break a tie by the initial, A to Z. Leave the data frame from step 7 intact. (8 points)

The five initials are `M.S.` (117), `M.B.` (100), `M.M.` (94), `A.M.` (89), and `M.P.` (84). The periods are part of the string. Missing both periods is a 1-point deduction. Overwriting `simple` so that step 9 sees only these five rows is a 3-point deduction.

```
simple |>
  mutate(
    initials = str_c(
      str_sub(first_name, 1, 1), ".",
      str_sub(family_name, 1, 1), "."
    )
  ) |>
  count(initials) |>
  arrange(desc(n), initials) |>
  slice_head(n = 5)
```

```
# A tibble: 5 x 2
  initials     n
  <chr>    <int>
1 M.S.       117
2 M.B.       100
3 M.M.        94
4 A.M.        89
5 M.P.        84
```

9. From the data frame in step 7, keep rows whose `first_name` starts with `M` and whose `family_name` has at least 8 characters. Use `str_starts()` and `str_length()`. (8 points)

`str_starts(first_name, "M")` and `str_length(family_name) >= 8`. Doing only one of the two filters is 4 points.

```
kept <- simple |>
  filter(str_starts(first_name, "M"), str_length(family_name) >= 8)
```

10. Unite `first_name` and `family_name` into `player_name`, with one space between them. Keep `player_api_id` and `player_name` only. Assign the result to `df1`, and print how many rows it has. (6 points)

`df1` has 373 rows and two columns, `player_api_id` and `player_name`. A hand-built `str_c(first_name, " ", family_name)` is full credit in place of `unite()`.

```
df1 <- kept |>
  unite(player_name, first_name, family_name, sep = " ") |>
  select(player_api_id, player_name)

nrow(df1)
```

```
[1] 373
```

11. From `Player_Attributes`, group by `player_api_id`. For each player, compute `potential_avg`, the mean of `potential`, and `sd_speed`, the standard deviation of `sprint_speed`. Ignore missing values. Assign the result to `df2`. (10 points)

One row per `player_api_id`, for every player in `Player_Attributes`, not only the 373 names. Restricting the summary to those 373 before the means is full credit, because the join drops the rest. Dropping `na.rm = TRUE` is a 2-point deduction.

```
df2 <- Player_Attributes |>
  group_by(player_api_id) |>
  summarise(
    potential_avg = mean(potential, na.rm = TRUE),
    sd_speed = sd(sprint_speed, na.rm = TRUE),
    .groups = "drop"
  )
```

12. Left join `df1` to `df2` by `player_api_id`. Drop a row when `potential_avg` or `sd_speed` is missing. Print how many rows remain. (8 points)

373 rows. None of these 373 have a missing mean or a missing standard deviation, so the filter does not drop a point. `inner_join()` gives the same 373 rows and is full credit here. A join that multiplies rows, because the attributes were not summarised first, is a 4-point deduction.

```
joined <- df1 |>
  left_join(df2, by = "player_api_id") |>
  filter(!is.na(potential_avg), !is.na(sd_speed))

nrow(joined)
```

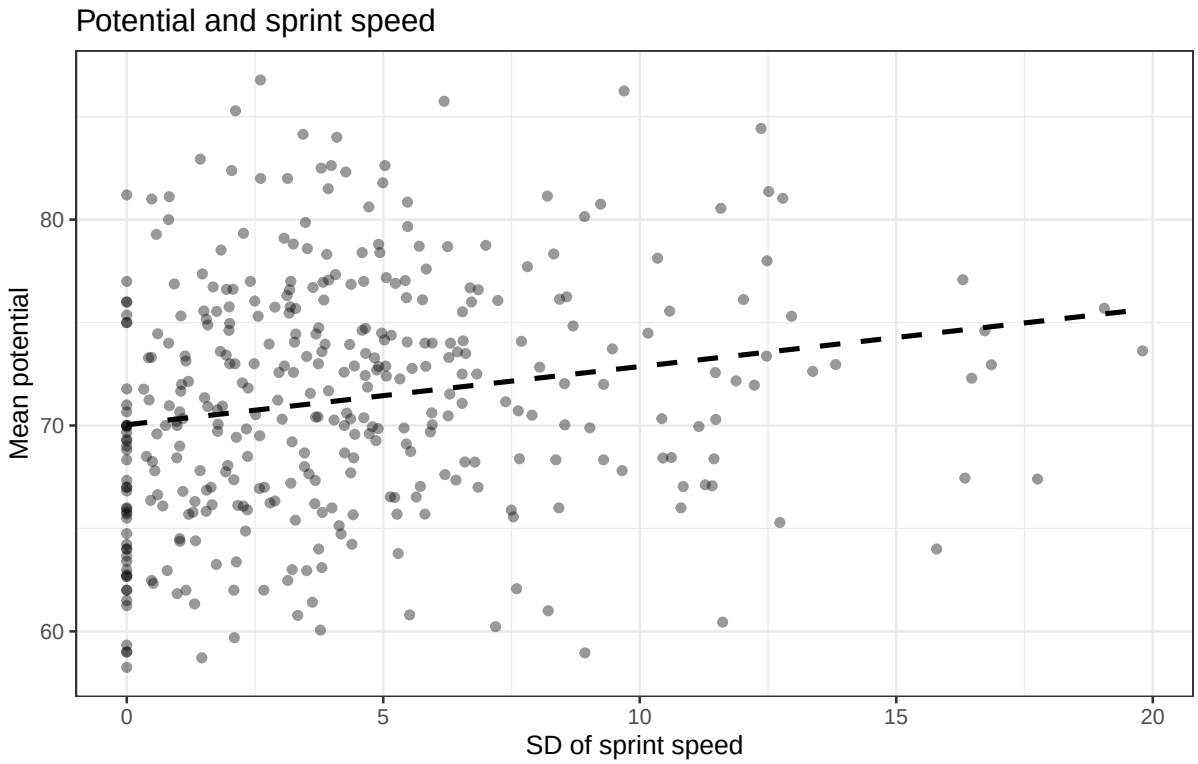
```
[1] 373
```

13. Plot `sd_speed` on the horizontal axis and `potential_avg` on the vertical axis. Use points with an alpha so overlapping points stay visible, and one straight smooth in black, dashed, with no gray band. Use `theme_bw()`. Title the plot `Potential and sprint speed`. Label the axes `SD of sprint speed` and `Mean potential`. In one sentence, say how mean potential changes as the standard deviation of sprint speed increases. (10 points)

The smooth is `method = "lm"`, `se = FALSE`, black, and dashed. A loess curve is a 1-point deduction. A gray band is a 1-point deduction. `theme_bw()`, the title, and the two axis labels are required.

The dashed line rises slightly. Mean potential is only a little higher when the standard deviation of sprint speed is higher, and the cloud is wide. A weak positive relationship is full credit for the sentence.

```
ggplot(joined, aes(x = sd_speed, y = potential_avg)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE, color = "black", linetype = "dashed") +
  theme_bw() +
  labs(
    x = "SD of sprint speed",
    y = "Mean potential",
    title = "Potential and sprint speed"
  )
```



14. Write a function of one argument, `name`. If `name` is not a character vector of length 1, or it is missing, stop with the message `name must be one string`. Collapse the whitespace with `str_squish()`. If a comma appears, drop the comma and everything after it with `str_remove()`, then squish again. If the result contains no space, return it unchanged. Otherwise return the last word, with `str_extract()` and the pattern `\\S+$`. Run it on `"  Luis  Martinez "`, `"Abdoulaye Diallo Sadio,22"`, `"Luisinho"`, and `c("Aaron", "Taylor")`. This chunk has `#| error: true`, so the last call can stop and the document still renders. Leave that error visible. (10 extra credit)

The name is not graded. `last_word` below is one acceptable name. The four results are `Martinez`, `Sadio`, `Luisinho`, and the stop message. The function still has to reject a missing string.

```
last_word <- function(name) {
  if (!is.character(name) || length(name) != 1 || is.na(name)) {
    stop("name must be one string")
  }
  name <- str_squish(name)
  name <- str_remove(name, ",.*")
  name <- str_squish(name)
  if (!str_detect(name, " ")) {
```

```
    return(name)
  }
  str_extract(name, "\\S+$")
}

last_word("  Luis   Martinez ")
```

```
[1] "Martinez"
```

```
last_word("Abdoulaye Diallo Sadio,22")
```

```
[1] "Sadio"
```

```
last_word("Luisinho")
```

```
[1] "Luisinho"
```

```
last_word(c("Aaron", "Taylor"))
```

```
Error in `last_word()`:
! name must be one string
```